

WiFi Connection

I. Experiment Objective

Learn the Wi-Fi networking function of the MicroROS-V2 control board. Through this experiment, you will master how to configure the ESP32 as a STA (Station) to connect to a router, handle connection and disconnection events, and configure the Wi-Fi SSID and password through the Kconfig menu.

II. Experiment Principle

1. Keywords

Keyword	Description
STA (Station)	Station mode. The ESP32 acts as a client that connects to a wireless router. It is the most commonly used networking mode for IoT devices.
AP (Access Point)	Access Point mode. The ESP32 acts as a hotspot for other devices to connect to.
SSID	Service Set Identifier, the wireless network name, used to identify different Wi-Fi networks.
DHCP	Dynamic Host Configuration Protocol, which automatically assigns network parameters such as the IP address, subnet mask, and gateway to devices.
TCP/IP protocol stack	Transmission Control Protocol/Internet Protocol stack. The ESP32 implements network communication through the open-source lwIP protocol stack.
Kconfig	The configuration system of ESP-IDF, used to configure project parameters through the menuconfig menu interface.

2. Technical Architecture

This example implements the STA-mode Wi-Fi networking function of the ESP32. After startup, the program initializes NVS Flash and the network interface, configures the Wi-Fi as STA mode and sets the SSID and password (read from Kconfig), starts Wi-Fi, and then automatically connects to the router through the event handling mechanism. After obtaining an IP address, it prints the connection information.

Main component files:

- `main/main.c` — Program entry, calls the Wi-Fi initialization
- `components/wifi_sta/wifi_sta.c` — Wi-Fi STA mode implementation
- `components/wifi_sta/wifi_sta.h` — Wi-Fi initialization function declarations
- `components/wifi_sta/Kconfig.projbuild` — menuconfig configuration item definitions

III. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	Required
USB data cable	Required
2.4GHz wireless router (Wi-Fi hotspot)	Required

2. Prerequisites

1. Set up the ESP32-IDF development environment
2. Connect the MicroROS-V2 control board to the computer via USB

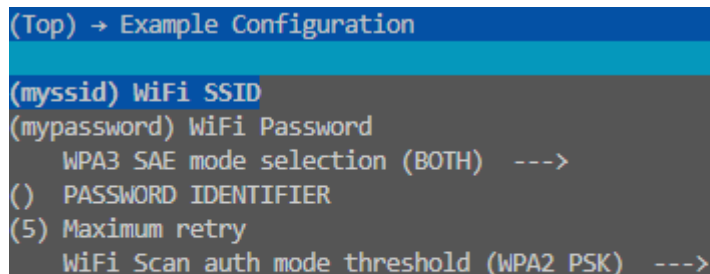
3. Compile and Flash

Before compiling, you need to configure the Wi-Fi SSID and password through menuconfig:

```
cd ~/esp/base_samples/wifi_sta
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```

In the menu, go to `Example Configuration` and set the following two items:

- **WiFi SSID:** enter your router name
- **WiFi Password:** enter your Wi-Fi password



```
(Top) → Example Configuration
(myssid) WiFi SSID
(mypassword) WiFi Password
WPA3 SAE mode selection (BOTH) --->
( ) PASSWORD IDENTIFIER
(5) Maximum retry
WiFi Scan auth mode threshold (WPA2 PSK) --->
```

After modifying, press S to save, then press Q to exit the configuration tool.

After configuration, compile and flash:

```
cd ~/esp/base_samples/wifi_sta
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

4. Stopping Method

In the terminal, press `Ctrl+J` to exit the serial monitor.

IV. Experiment Result

After successful compilation and flashing, the Monitor will output:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
I (xxx) WIFI_STA: ESP_WIFI_MODE_STA
I (xxx) WIFI_STA: wifi_init_sta finished.
I (xxx) WIFI_STA: got ip:192.168.x.xxx
I (xxx) WIFI_STA: connected to ap SSID:your_ssid password:your_password
```

The control board successfully connects to the router and obtains an IP address, indicating that the Wi-Fi networking function works normally.

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/wifi_sta/main/main.c`
- WiFi STA: `~/esp/base_samples/wifi_sta/components/wifi_sta/wifi_sta.h`,
`wifi_sta.c`
- Kconfig: `~/esp/base_samples/wifi_sta/components/wifi_sta/Kconfig.projbuild`

1. Main Program Logic

`main/main.c` is the program entry. After printing a welcome message, it calls `wifi_Net_Init()` to initialize Wi-Fi:

```
#include "wifi_sta.h"

void app_main(void)
{
    printf("hello yahboom\n");
    ESP_LOGI(TAG, "Nice to meet you!");
    wifi_Net_Init();
}
```

2. Wi-Fi Initialization Flow

The `wifi_Net_Init()` function in `components/wifi_sta/wifi_sta.c` completes the full initialization flow:

1. **Initialize NVS Flash** — Call `nvs_flash_init()` to save the Wi-Fi configuration information
2. **Call `wifi_init_sta()`** — executes the following steps:
 - Create the event group `s_wifi_event_group` to notify the connection result
 - Call `esp_netif_init()` to initialize the network interface
 - Call `esp_event_loop_create_default()` to create the default event loop
 - Call `esp_netif_create_default_wifi_sta()` to create the default STA network interface

- Call `esp_wifi_init()` to initialize the Wi-Fi driver
- Register the Wi-Fi event and IP event handlers
- Configure the Wi-Fi as STA mode and set the SSID and password
- Call `esp_wifi_start()` to start Wi-Fi

3. Event Handling Mechanism

The `event_handler()` function handles three types of events:

Event	Handling Logic
<code>WIFI_EVENT_STA_START</code>	Automatically calls <code>esp_wifi_connect()</code> after Wi-Fi starts to connect to the router
<code>WIFI_EVENT_STA_DISCONNECTED</code>	After disconnection, checks the retry count; if the maximum retry count has not been exceeded, reconnects
<code>IP_EVENT_STA_GOT_IP</code>	After obtaining an IP address, prints the IP information and sets the success flag

The maximum retry count is configured through `CONFIG_ESP_MAXIMUM_RETRY` (default 5). If the connection still fails after exceeding the retry count, the failure flag `WIFI_FAIL_BIT` is set.

4. Wi-Fi Configuration Parameters

The Wi-Fi configuration struct is defined as follows:

```
wifi_config_t wifi_config = {
    .sta = {
        .ssid = EXAMPLE_ESP_WIFI_SSID,          // Read from menuconfig
        .password = EXAMPLE_ESP_WIFI_PASS,      // Read from menuconfig
        .threshold.authmode = ESP_WIFI_SCAN_AUTH_MODE_THRESHOLD,
        .sae_pwe_h2e = ESP_WIFI_SAE_MODE,
        .sae_h2e_identifier = EXAMPLE_H2E_IDENTIFIER,
    },
};
```

5. Kconfig Configuration Items

Parameter definition file:

```
~/esp/base_samples/wifi_sta/components/wifi_sta/kconfig.projbuild
```

`components/wifi_sta/kconfig.projbuild` defines the following parameters that can be configured through `idf.py menuconfig`:

Configuration Item	Default Value	Description
<code>CONFIG_ESP_WIFI_SSID</code>	"mysid"	Wi-Fi network name
<code>CONFIG_ESP_WIFI_PASSWORD</code>	"mypassword"	Wi-Fi password
<code>CONFIG_ESP_MAXIMUM_RETRY</code>	5	Maximum retry count after connection failure
<code>CONFIG_ESP_WIFI_AUTH_MODE</code>	WPA2_PSK	Wi-Fi authentication mode threshold (supports OPEN / WEP / WPA_PSK / WPA2_PSK / WPA3_PSK, etc.)
<code>CONFIG_ESP_WPA3_SAE_PWE_MODE</code>	BOTH	WPA3 SAE mode (HUNT_AND_PECK / H2E / BOTH)

6. Waiting for the Connection Result

`wifi_init_sta()` blocks and waits for the connection result through `xEventGroupwaitBits()`:

```
EventBits_t bits = xEventGroupwaitBits(s_wifi_event_group,
    WIFI_CONNECTED_BIT | WIFI_FAIL_BIT,
    pdFALSE, pdFALSE, portMAX_DELAY);
```

- If `WIFI_CONNECTED_BIT` is set, the connection succeeded
- If `WIFI_FAIL_BIT` is set, the connection failed after exceeding the retry count

7. Key Components

Function	Description
<code>wifi_Net_Init()</code>	Initializes NVS and starts the Wi-Fi connection flow
<code>wifi_init_sta()</code>	Configures and enables STA-mode Wi-Fi, blocking until the connection result is obtained
<code>event_handler()</code>	Handles the Wi-Fi and IP event callbacks

VI. Common Problems

Problem	Cause	Solution
Cannot scan the router	Incorrect SSID configuration or too weak router signal	Check whether the SSID is correct in menuconfig and make sure the router is within signal range
Connection failure with password error	Incorrect Wi-Fi password configuration	Re-enter the correct Wi-Fi password in menuconfig
WPA3 connection failure	The router does not support WPA3 or the configuration does not match	Change the authentication mode to WPA2_PSK in menuconfig
Still cannot connect after many retries	Router incompatibility or too far away	Check whether the router works on the 2.4GHz band; the ESP32 does not support 5GHz